

Meta Clustering Script Update Brief (#1)

Old: meta_ANARCI_clustering_alt.py

New: meta_ANARCI_clustering_alt_edit.py

Runtime Statistics

Runtime on cluster with a 10k obs. dataset averaged over 50 runs:

- meta_ANARCI_clustering_alt: 57.16 seconds
- meta_ANARCI_clustering_alt_edit: 52.70 seconds
- ***4.46 second improvement***

Summary of Changes

Overview

This first update improves performance through several optimizations. It introduces dynamic chunking based on CPU count for parallel processing, replaces iterative operations with vectorized pandas operations, and uses the `imap_unordered` pattern with `itertools.repeat` to minimize memory consumption during parallel tasks. The code eliminates dataframe copies inside loops, replacing them with a single copy outside the loop, and implements efficient data filtering using boolean indexing rather than multiple selection operations. In the clustering logic, it uses `iloc` for direct integer-based indexing instead of label-based `loc` operations and checks if columns exist before assignment to avoid expensive column creation operations. These optimizations collectively reduce memory overhead and processing time, particularly for large datasets.

Specifics

1. Dynamic Chunking + `imap_unordered` with `itertools.repeat` (line 81-97) (parallel translation)

- Adaptive chunk sizing allows it to scale with data size and CPU availability. In the original script, constant chunk sizing was inefficient because it either caused too many hand-offs between cores or CPU idling when one was overloaded.
- *imap_unordered* allows results from faster cores to return without waiting on slower ones, which speeds up execution especially when some sequences take longer to translate.
- The old version created a big list of tuples, pairing each DNA sequence with the same fixed parameters (like start/end motifs and scan limits). This used more memory than necessary because those parameters don't change (set once when the script runs). The new version uses *itertools.repeat* to supply those fixed values only when needed, without

building the whole list. This reduces memory use and works better with `imap_unordered`, which lets workers return results as they finish, speeding up the parallel processing step.

2. Vectorized operations and boolean indexing (line 103-110) (filtering bad sequences)

- The old version filtered sequences by computing separate masks (e.g., for stop codons or low motif matches) using `.map` with `lambda` functions, which are slow and memory-intensive. This created multiple intermediate variables and required several steps. The new version replaces this with a single vectorized expression using pandas' built-in string and numeric filters. This avoids Python loops, uses less memory, and applies the filters in one fast pass over the data.

3. *iloc* instead of *loc* (229-240) (merging labels onto dataframe)

- The old version used `loc` to assign clone labels to DataFrame rows, which is slower because pandas has to map labels to positions each time. The new version uses `iloc` with precomputed row and column positions, allowing for faster, direct access. This avoids repeated index lookups and speeds up assignment during clustering, especially for large datasets.

4. Left instead of outer merges (134, 310, 344) (merging with ANARCI annotations, VHH metadata, and CDR region assignments)

- The old version used outer merges, which keep all rows from both DataFrames and can introduce extra rows filled with missing values. This increases memory use and slows things down. The new version switches to left joins, which preserve only the relevant rows from the main dataset and bring in matching information as needed. This keeps the data clean, improves merge performance, and avoids downstream errors from unexpected NaNs.

5. Remove redundant copy (239 in og) (main clustering loop)

- Removed unnecessary `.copy` call after label assignment. In the old version, this was used to defragment the DataFrame after multiple insertions, but the updated code uses safe `iloc`-based assignments instead. Skipping the copy avoids duplicating a large DataFrame in memory, reducing overhead and speeding up the pipeline.